
Competitive Programmer's Handbook

Commentary on Antti Laaksonen's Guide

Andre Ruiz Loera

Preface

Contents

Preface	3
1 Introduction	7
2 Time Complexity	9
3 Sorting	11
4 Data Structures	13
5 Dynamic Programming	15
6 Graph Algorithms	17
7 Algorithm Design Topics	19
8 Range Queries	21
9 Tree Algorithms	23
10 Mathematics	25
11 Advanced Graph Algorithms	27
12 Geometry	29
13 Directed Graphs	31
14 Strong Connectivity	33

15 Tree Queries	35
16 Paths and Circuits	37
17 Flows and Cuts	39
18 String Algorithms	41
19 Square Root Algorithms	43
20 Segment Trees Revisited	45
21 Combinatorics	47
22 Matrix Operations	49
23 Probability	51
24 Game Theory	53
25 Number Theory	55
26 String Matching	57
27 Advanced Topics	59

Chapter 1

Introduction

One thing this chapter makes clear right away is that competitive programming is not just about coming up with the right algorithm. You also have to write it correctly. It sounds obvious, but it is easy to forget when you are in the middle of a contest trying to think through the logic. The judges do not care about your idea. They care about whether the program actually works and does it fast enough. Speed matters in another way too. You have to write code quickly. There is no time to obsess over style or elegance. You solve, you implement, and you move on. Most people in competitive programming use C++. There is a simple reason for that. C++ is fast and the standard library gives you countless useful tools that you do not have to reinvent. You can get a priority queue, a sorting function, or a hash map whenever you need it. Even so, the book reminds us that the problems are usually written so that no language has a special advantage. A good algorithm will beat a bad one regardless of the language. One thing I appreciate about the C++ standard library is that you really do get almost everything you need right there. You do not have to pull in extra libraries or set up anything complicated. It is all already waiting for you. There are also little tricks that help with input and output speed. For example, writing `"\n"` is usually faster than `endl` because `endl` flushes the buffer every time. Most of the time you do not need that flush, so `endl` just slows you down. `scanf` and `printf` can also be faster than `cin` and `cout`, although you can adjust settings to speed up the C++ streams too. `getline` is useful when you need to read a full line that contains spaces, since the normal `cin` stops at spaces. Sometimes you do not know how much input you will get. In those cases, using `cin >> x` repeatedly lets you read everything until the input ends. It is a simple trick, but it comes up often. The

book also mentions `freopen`, which is handy when you want to test your program on local files. For example:

```
1 freopen("input.txt", "r", stdin);  
2 freopen("output.txt", "w", stdout);
```

After that, all your input comes from `input.txt` and everything you print goes to `output.txt`. It makes testing much easier. Finally, the chapter touches on something every competitor eventually learns the hard way. The `int` type does not always have enough range. When values start getting large, you have to switch to `long long` or your answer will overflow and you will lose points for a mistake you did not notice. Overall, this first chapter sets the tone. Competitive programming is part problem solving, part careful coding, and part learning small habits that save time and avoid mistakes. It is not only about being clever. It is about being precise and practical under pressure.

Chapter 2

Time Complexity

Chapter 3

Sorting

Chapter 4

Data Structures

Chapter 5

Dynamic Programming

Chapter 6

Graph Algorithms

Chapter 7

Algorithm Design Topics

Chapter 8

Range Queries

Chapter 9

Tree Algorithms

Chapter 10

Mathematics

Chapter 11

Advanced Graph Algorithms

Chapter 12

Geometry

Chapter 13

Directed Graphs

Chapter 14

Strong Connectivity

Chapter 15

Tree Queries

Chapter 16

Paths and Circuits

Chapter 17

Flows and Cuts

Chapter 18

String Algorithms

Chapter 19

Square Root Algorithms

Chapter 20

Segment Trees Revisited

Chapter 21

Combinatorics

Chapter 22

Matrix Operations

Chapter 23

Probability

Chapter 24

Game Theory

Chapter 25

Number Theory

Chapter 26

String Matching

Chapter 27

Advanced Topics